Vous : cryptage chiffrement RSA en python

ChatGPT :

Le chiffrement RSA est un algorithme de chiffrement asymétrique qui utilise une paire de clés : une clé publique (pour le chiffrement) et une clé privée (pour le déchiffrement). Voici un exemple de code Python pour implémenter le chiffrement et le déchiffrement RSA à l'aide de la bibliothèque `pycryptodome`, qui est une bibliothèque couramment utilisée pour la cryptographie en Python.

Étapes principales :

1. Générer une paire de clés (publique et privée).
2. Utiliser la clé publique pour chiffrer un message.
3. Utiliser la clé privée pour déchiffrer le message.

Installation de la bibliothèque :

Vous devez d'abord installer `pycryptodome` via pip si vous ne l'avez pas déjà :

```
pip install pycryptodome
```

Exemple de code RSA :

```
from Crypto.PublicKey import RSA

from Crypto.Cipher import PKCS1_OAEP

from Crypto.Random import get_random_bytes

import base64
```

```
# Générer une paire de clés (publique et privée)
key = RSA.generate(2048)  # Génère une clé de 2048 bits
private_key = key.export_key()  # Clé privée
public_key = key.publickey().export_key()  # Clé publique

# Affichage des clés pour référence (optionnel)
print("Clé privée:\n", private_key.decode())
print("Clé publique:\n", public_key.decode())

# Fonction pour chiffrer un message avec la clé publique
def encrypt_message(public_key, message):
    pub_key = RSA.import_key(public_key)
    cipher = PKCS1_OAEP.new(pub_key)
    encrypted_message = cipher.encrypt(message.encode())
    return base64.b64encode(encrypted_message)  # Encodage pour rendre le message lisible

# Fonction pour déchiffrer un message avec la clé privée
def decrypt_message(private_key, encrypted_message):
    priv_key = RSA.import_key(private_key)
```

```
    cipher = PKCS1_OAEP.new(priv_key)

    decoded_message = base64.b64decode(encrypted_message)

    decrypted_message = cipher.decrypt(decoded_message)

    return decrypted_message.decode()


# Exemple d'utilisation

message = "Bonjour, voici un message secret!"

print("Message original:", message)


# Chiffrement avec la clé publique

encrypted_message = encrypt_message(public_key, message)

print("Message chiffré:", encrypted_message.decode())


# Déchiffrement avec la clé privée

decrypted_message = decrypt_message(private_key, encrypted_message)

print("Message déchiffré:", decrypted_message)
```

Explication du code :

1. **Génération des clés** : Le code génère une paire de clés RSA (publique et privée) de 2048 bits. La clé privée et la clé publique sont exportées sous forme de chaînes.

2. **Chiffrement** : Le message est d'abord encodé en utilisant la clé publique. Le chiffrement est effectué en utilisant l'algorithme `PKCS1_OAEP`, qui est un des modes de chiffrement RSA sécurisé.
3. **Déchiffrement** : Le message chiffré est déchiffré en utilisant la clé privée avec le même algorithme `PKCS1_OAEP`.

Remarques importantes :

- RSA est un algorithme de chiffrement asymétrique, ce qui signifie qu'une clé publique peut être utilisée pour chiffrer un message, mais seul le détenteur de la clé privée correspondante pourra le déchiffrer.
- RSA est souvent utilisé pour échanger des clés secrètes pour un chiffrement symétrique (comme AES), car RSA est plus lent pour de grandes quantités de données.

Ce code est un exemple de base et peut être étendu pour des applications plus complexes, comme la signature numérique et l'authentification.